

14. 上位 MPC 距離メッシュ生成

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/upper_horizon.py
種別	Python
区分	planner helper
役割	固定または適応距離メッシュを作り、現在地点から先の control point と segment を決める。
起動文脈	distance-domain upper planner の最初の一步。

このファイルを読む理由

固定または適応距離メッシュを作り、現在地点から先の control point と segment を決める。

- build_upper_distance_horizon が中心。
- plan_segment_index が現在位置から有効速度区間を引く。

呼び出し関係

どこから呼ばれるか

- mpc_node.py
- scripts/solar_sim.py

次に読むと理解が進むファイル

- mpc_solarcar/upper_policy.py

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
@dataclass
class UpperDistanceHorizon:
    ds_seq_km: np.ndarray
    seg_s_km: np.ndarray
    ctrl_s_km: np.ndarray

    @property
    def total_km(self) -> float:
        return float(np.sum(self.ds_seq_km)) if len(self.ds_seq_km) else 0.0

def _weighted_extra(count: int, growth: float) -> np.ndarray:
    if count <= 1:
        return np.ones(max(1, count), dtype=float)
    growth = max(1.0, float(growth))
    if abs(growth - 1.0) <= 1.0e-9:
        return np.ones(count, dtype=float)
    return np.power(growth, np.arange(count, dtype=float))
```

主要構造の読み取り

主要クラスは UpperDistanceHorizon。主要関数は total_km, build_upper_distance_horizon, plan_segment_index。

主要クラス・関数

行	種別	名前	補足
14	class	UpperDistanceHorizon	
20	function	total_km	args: self
24	function	_weighted_extra	args: count, growth
33	function	_adaptive_ds_sequence	args: target_km
82	function	build_upper_distance_horizon	
136	function	plan_segment_index	args: plan_segments, s_km

ファイルを上から読んだときの定義順

行	内容
10	MIN_DISTANCE_STEP_KM に 1e-06 の結果を代入する。
14	クラス UpperDistanceHorizon を定義する。
24	関数 _weighted_extra を定義する。
33	関数 _adaptive_ds_sequence を定義する。
82	関数 build_upper_distance_horizon を定義する。
136	関数 plan_segment_index を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>import math</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L48, L112。
4	<code>from dataclasses import dataclass</code>	<code>dataclasses</code> から <code>dataclass</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L13。
5	<code>from typing import Iterable, List</code>	<code>typing</code> から <code>Iterable</code> 、 <code>List</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L136, L137。
7	<code>import numpy as np</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L15, L16, L17, L21, L24, L26, L29, L30, ...。

関数・クラスを上から順に解説

L14 クラス `UpperDistanceHorizon`

- 定義: `UpperDistanceHorizon(bases=())`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `len`, `sum`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。

1. `ds_seq_km` にを代入する。
2. `seg_s_km` にを代入する。
3. `ctrl_s_km` にを代入する。
4. 関数 `total_km` を定義する。

L24 関数 `_weighted_extra`

- 定義: `_weighted_extra(count, growth)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `arange`, `float`, `max`, `ones`, `power`
- 戻り値の要点: `np.power(growth, np.arange(count, dtype=float)) / np.ones(max(1, count), dtype=float) / np.ones(count, dtype=float)`

1. 条件 `count <= 1` を判定し、真なら内部処理を行う。
2. `growth` に `max(1.0, float(growth))` の結果を代入する。
3. 条件 `abs(growth - 1.0) <= 1e-09` を判定し、真なら内部処理を行う。
4. `np.power(growth, np.arange(count, dtype=float))` を返す。

L33 関数 `_adaptive_ds_sequence`

- 定義: `_adaptive_ds_sequence(target_km, max_steps, min_ds_km, max_ds_km, growth)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_weighted_extra`, `any`, `array`, `ceil`, `copy`, `enumerate`, `flatnonzero`, `float`, `full`, `int`, `max`, `maximum`
- 戻り値の要点: `ds / np.array([target_km], dtype=float) / np.array([target_km], dtype=float) / np.full(step_count, target_km / step_count, dtype=float)`

1. `target_km` に `max(MIN_DISTANCE_STEP_KM, float(target_km))` の結果を代入する。
2. `min_ds_km` に `max(MIN_DISTANCE_STEP_KM, float(min_ds_km))` の結果を代入する。
3. 条件 `max_steps <= 1` を判定し、真なら内部処理を行う。
4. 条件 `target_km <= min_ds_km` を判定し、真なら内部処理を行う。
5. `step_count` に `int(min(max_steps, max(1, math.ceil(target_km / min_ds_km))))` の結果を代入する。
6. `base` に `np.full(step_count, min_ds_km, dtype=float)` の結果を代入する。
7. `base_sum` に `float(base.sum())` の結果を代入する。
8. 条件 `target_km <= base_sum + 1e-09` を判定し、真なら内部処理を行う。
9. `weights` に `_weighted_extra(step_count, growth)` の結果を代入する。
10. `weights` に `weights / max(float(weights.sum()), 1e-09)` の結果を代入する。

L82 関数 `build_upper_distance_horizon`

- 定義: `build_upper_distance_horizon(mode, s0_km, race_km, ds_km, horizon_km, max_steps, ctrl_km, adaptive_min_ds_km, adaptive_max_ds_km, adaptive_growth)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `UpperDistanceHorizon`, `_adaptive_ds_sequence`, `append`, `arange`, `array`, `ceil`, `concatenate`, `cumsum`, `float`, `full`, `int`, `len`
- 戻り値の要点: `UpperDistanceHorizon(ds_seq_km=ds_seq, seg_s_km=seg_s, ctrl_s_km=ctrl_s)`

1. `ds_km` に `max(MIN_DISTANCE_STEP_KM, float(ds_km))` の結果を代入する。
2. `horizon_km` に `max(ds_km, float(horizon_km))` の結果を代入する。
3. `remaining_km` に `max(0.0, float(race_km) - float(s0_km))` の結果を代入する。
4. `max_steps` に `max(1, int(max_steps))` の結果を代入する。
5. `mode` に `str(mode or 'fixed').strip().lower()` の結果を代入する。
6. 条件 `mode in {'adaptive_full_race', 'remaining_race', 'full_race'}` を判定し、真なら内部処理を行う。
7. `seg_s` に `np.concatenate([[0.0], np.cumsum(ds_seq[:-1], dtype=float)])` の結果を代入する。
8. `total_km` に `float(np.sum(ds_seq))` の結果を代入する。
9. `control_end_km` に `float(seg_s[-1]) if len(seg_s) else 0.0` の結果を代入する。
10. `ctrl_step_km` に `float(ctrl_km) if ctrl_km and ctrl_km > 0.0 else float(ds_seq[0])` の結果を代入する。

L136 関数 `plan_segment_index`

- 定義: `plan_segment_index(plan_segments, s_km)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `enumerate`, `float`, `get`, `len`, `list`
 - 戻り値の要点: `len(segments) - 1 / -1 / idx`
1. `segments` に `list(plan_segments)` を代入する。
 2. 条件 `not segments` を判定し、真なら内部処理を行う。
 3. `s_km` に `float(s_km)` の結果を代入する。
 4. `enumerate(segments)` を順に走査し、各要素を `(idx, seg)` に入れて処理する。
 5. `len(segments) - 1` を返す。

処理の流れ

1. ソース中の主要関数を通じて処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/upper_horizon.py` を **planner helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。